

Rote

Qwen3-VL on the ASUS GX10

A working guide to ai.py — Python API, tool calls, and commands

Host	gx10-3d18 (100.88.242.48)
Reached over	Tailscale – works from any machine on the tailnet
Server	Ollama 0.32.9, port 11434
Model	qwen3-vl:30b – 30.5B MoE, Q4_K_M, 19.6 GB
Capabilities	completion, vision, tools, thinking
Client	ai.py – library, requests only
Entry point	main.py – connects, warms, hands over the client

1. How it fits together

Inference runs entirely on the GX10. Your project holds no model weights and needs no GPU: `ai.py` is an HTTP client that posts to Ollama's API on the GX10 and streams results back.

The two machines are joined by Tailscale, so the GX10 is addressable as `gx10-3d18` from anywhere — home, a cafe, a different network — with no port forwarding, no public IP, and no VPN configuration. Ollama itself is unauthenticated; the tailnet is the security boundary.

your project		ASUS GX10
+-----+		+-----+
from ai	Tailscale	Ollama :11434
import ai	=====>	qwen3-vl:30b (19.6GB)
ai.ask(...)	HTTP/JSON	nomic-embed-text
+-----+		+-----+

Installing

Only one dependency, and it is almost certainly already present:

```
pip install requests
```

Confirm the link works before anything else:

```
$ python ai.py --check
host    http://gx10-3d18:11434
status  up
model   qwen3-vl:30b
loaded  qwen3-vl:30b
```

Where it plugs in

`ai.py` is a library. `main.py` is the entry point and owns startup: it connects, fails loudly if the GX10 is unreachable or the model is missing, and warms the model so the first real request does not pay the load cost.

```
# main.py
from ai import AI, AIError

def setup_ai() -> AI:
    ai = AI()
    if not ai.is_up():
        raise AIError(f"GX10 unreachable at {ai.host}")
    if not ai.has_model():
        raise AIError(f"{ai.model} is not on the GX10. Run: python ai.py --pull")
    ai.warm()
    return ai
```

Anywhere else in the project, take the client that `main.py` built, or import the module-level one:

```
from ai import ai          # already configured from the environment
```

The `python ai.py ...` commands in section 4 are a convenience for checking the link, pulling, and quick questions. They are not how the project runs — that is `python main.py`.

Configuration

Every default is overridable by environment variable, so deployment needs no code edits:

Variable	Default	Meaning
ROTE_AI_HOST	<code>http://gx10-3d18:11434</code>	Ollama endpoint on the GX10
ROTE_AI_MODEL	<code>qwen3-vl:30b</code>	Model tag used by default
ROTE_AI_TIMEOUT	<code>600</code>	Per-request timeout, seconds
ROTE_AI_KEEP_ALIVE	<code>30m</code>	How long the model stays in GPU memory

Or set them per instance, which is how you reach a second model:

```
from ai import AI

vl    = AI()                                # qwen3-vl:30b
coder = AI(model="qwen3-coder:30b")         # a different model
hot   = AI(keep_alive="2h", options={"temperature": 0.2})
```

Cold starts. Loading 19.6 GB into GPU memory takes roughly 30 seconds. The first call after an idle period pays that cost, and a request that lands mid-load can return a 5xx — `ai.py` retries those automatically. Call `ai.warm()` at startup, or raise `keep_alive`, to avoid the wait entirely.

2. The Python API

Import the ready-made client for the common case, or construct your own:

```
from ai import ai          # a module-level AI() using the env defaults
from ai import AI          # the class, when you want your own settings
```

ask() — one question, one answer

```
ai.ask("Summarise this log in one line.")
ai.ask("List three colours.", system="Answer tersely.")
ai.ask("Rate this 1-10.", temperature=0)      # options pass straight through
```

Any keyword you add is forwarded to Ollama as a generation option: `temperature`, `top_p`, `seed`, `num_ctx`, `num_predict`, `stop`.

stream() — tokens as they arrive

Returns a generator of text fragments. Use it for anything a person watches, so output starts appearing immediately instead of after the full generation:

```
for piece in ai.stream("Write a haiku about caching."):
    print(piece, end="", flush=True)
```

look() — the vision path

This is the reason for the VL model. Pass a file path, raw bytes, a `pathlib.Path`, or a base64 / data-URL string; `ai.py` encodes whatever you hand it.

```
ai.look("screenshot.png", "Which button is highlighted?")
ai.look(Path("receipt.jpg"), "Extract the total as a number.")
ai.look(png_bytes, "Is there an error dialog on screen?")

# look() is a shorthand; ask() takes images too, including several at once
ai.ask("What changed between these?", images=["before.png", "after.png"])
```

Verified: a 400×200 PNG of a red circle and a blue square returned "The image contains a red circle and a blue square."

conversation() — multi-turn with memory

`ask()` is stateless. When the model needs to remember earlier turns, use a conversation, which keeps the message list for you:

```

chat = ai.conversation(system="You are terse.")
chat.say("Remember the number 7.")
chat.say("What number?")          # -> "seven"

for piece in chat.stream_say("Explain why."): # streaming, still remembered
    print(piece, end="")

chat.messages                     # the raw list, yours to inspect or persist
chat.reset()                     # clear history, keep the system prompt

```

Images work mid-conversation too: `chat.say("What is this?", images=["a.png"])`.

Structured output

Pass `format="json"` to force syntactically valid JSON, or a JSON Schema to pin the exact shape — far more reliable than asking for JSON in the prompt and hoping:

```

import json

raw = ai.ask("List three colours as {\n\"colours\": [...]}.", format="json")
data = json.loads(raw)

schema = {
    "type": "object",
    "properties": {
        "total": {"type": "number"},
        "currency": {"type": "string"},
    },
    "required": ["total", "currency"],
}
receipt = json.loads(ai.look("receipt.jpg", "Extract the total.", format=schema))

```

Thinking

Qwen3-VL is a reasoning model. Set `think=True` to let it reason before answering, or `think=False` to suppress it for speed. The reasoning lands in the response's thinking field, not in content, so `ask()` still returns just the answer.

```
ai.ask("Which of these two designs scales better, and why?", think=True)
```

embed() — vectors for retrieval

Embeddings use a small dedicated model rather than the 30B one, so they are fast and do not evict Qwen3-VL from GPU memory:

```
vectors = ai.embed(["first chunk", "second chunk"]) # 768 dimensions each
```

3. Tool calls

Tool calling lets the model run *your* Python functions and use the results. You describe what is available, the model decides what to call and with which arguments, `ai.py` executes it and feeds the return value back, and the model answers using real data.

Write a normal function

No decorators and no schema by hand. Two rules carry all the weight:

- **Annotate every parameter.** The annotation becomes the JSON type (`str`, `int`, `float`, `bool`, `list`, `dict`). A parameter with a default is optional; one without is required.
- **Write the docstring for the model.** Its first line is the tool description — this is how the model decides when to call it, so say plainly what it does.

```
import json

def get_weather(city: str, units: str = "celsius") -> str:
    """Look up the current weather for a city."""
    return json.dumps({"city": city, "temp": 14, "sky": "raining"})

def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b
```

Hand them to the model

```
answer = ai.with_tools(
    "What is the weather in Boston, and what is 21 + 21?",
    tools=[get_weather, add],
)
print(answer)
```

Which produces, with both tools actually executed:

```
CALL get_weather {'city': 'Boston'} -> {"city": "Boston", "temp": 14, "sky": "raining"}
CALL add {'a': 21, 'b': 21} -> 42

The current weather in Boston is 14 C and it is raining.
Additionally, 21 + 21 equals 42.
```

Watching what it does

`on_call` fires after each execution, which is the hook for logging, metrics, or a confirmation prompt before a destructive tool runs:

```
def log(name, arguments, result):
    print(f" {name}({arguments}) -> {result}")

ai.with_tools("...", tools=[...], on_call=log)
```

Vision and tools together

Tools compose with images, which is where this model earns its keep — it can read a screenshot and then act on what it saw:

```
ai.with_tools(
    "Read the order number off this screenshot and look up its status.",
    tools=[lookup_order],
    images=["screenshot.png"],
)
```

Rules the loop follows

Behaviour	What happens
Return values	Strings are passed through; anything else is JSON-encoded before being handed back to the model.
Exceptions	Caught and returned to the model as text, so it can correct its arguments and retry rather than crashing your program.
Unknown tool	Reported back as “No such tool” instead of raising.
Several calls	A single turn may call multiple tools; all run before the model continues.
max_rounds	Caps the call/answer cycles at 6 by default. Exceeding it raises <code>AIError</code> rather than looping forever.

Tools execute real code with model-chosen arguments. Treat a tool the way you would treat an HTTP handler taking untrusted input: validate arguments, scope filesystem and network access narrowly, and put anything destructive behind an `on_call` confirmation.

Inspecting or hand-writing a schema

`as_tool()` is the function that derives the schema. Call it directly to see what the model is being shown:

```

from ai import as_tool
print(as_tool(get_weather))

{'type': 'function',
 'function': {'name': 'get_weather',
              'description': 'Look up the current weather for a city.',
              'parameters': {'type': 'object',
                             'properties': {'city': {'type': 'string'},
                                             'units': {'type': 'string'}},
                             'required': ['city']}}}

```

For full control — enums, nested objects, descriptions per parameter — skip the helper and drive `chat()` yourself, dispatching `tool_calls` however you like:

```

result = ai.chat(messages, tools=[my_hand_written_schema])
for call in result["message"].get("tool_calls", []):
    name = call["function"]["name"]
    args = call["function"]["arguments"]
    ...

```

4. Command line

Running `ai.py` directly is a maintenance and scratch tool — checking the link, pulling the model, asking a one-off question. The project itself runs through `main.py`.

Command	What it does
<code>python ai.py "prompt"</code>	Ask a question; streams the answer to stdout
<code>python ai.py -i img.png "q"</code>	Ask about an image (repeatable for several)
<code>python ai.py --chat</code>	Interactive session with history
<code>python ai.py --check</code>	Connection, model, and what is loaded
<code>python ai.py --pull</code>	Download the model onto the GX10
<code>python ai.py --warm</code>	Preload into GPU memory (~30s)

Flags

Flag	Effect
<code>-s, --system</code>	System prompt
<code>-m, --model</code>	Use another model for this run
<code>--host</code>	Point at a different Ollama server
<code>-t, --temperature</code>	0 for deterministic, higher for varied
<code>--json</code>	Force valid JSON output
<code>--no-stream</code>	Wait and print the whole answer at once

Piping

With no prompt argument it reads stdin, so it composes with everything else:

```
git diff | python ai.py "Write a commit message for this diff."
cat error.log | python ai.py -s "You are a debugger." "What failed here?"
python ai.py --json "List 3 colours" | jq .
```

Interactive session

```
$ python ai.py --chat
qwen3-vl:30b on http://gx10-3d18:11434. Ctrl-D or /exit to quit, /reset to clear.

> what did I just ask about?
...
```

5. Operations

Keeping the model resident is the single biggest lever on latency:

```
ai.warm()          # load into GPU memory now, so the next call is instant
ai.is_up()         # True if the GX10 answers
ai.models()        # every model on the box
ai.loaded()        # what is currently in GPU memory
ai.has_model()     # is the configured model downloaded?
ai.pull()          # download it (safe to call when it already exists)
```

A long-running service should warm on startup and set a generous keep-alive:

```
ai = AI(keep_alive="24h")
if not ai.has_model():
    ai.pull()
ai.warm()
```

Other models on the GX10

The box hosts several; switch with `AI(model=...)` or `-m`. Note that loading a second large model may evict the first.

Model	Size	Good for
qwen3-vl:30b	19.6 GB	Vision, tools, reasoning — the default
qwen3:30b-a3b	18.6 GB	Text only, tools, thinking
qwen3-coder:30b	18.6 GB	Code generation and editing
qwen2.5-coder:32b	19.9 GB	Code, including fill-in-the-middle
deepseek-r1:70b	42.5 GB	Heavy reasoning; slow
qwen3:8b	5.2 GB	Fast text and tools
nomic-embed-text	0.3 GB	Embeddings (768-dim)

6. Using the OpenAI SDK instead

Ollama also serves an OpenAI-compatible API, so any library that speaks that protocol — the OpenAI SDK, LangChain, LlamaIndex, Vercel AI SDK — can point straight at the GX10. You do not need `ai.py` for this.

```
from openai import OpenAI

client = OpenAI(base_url="http://gx10-3d18:11434/v1", api_key="ollama")

client.chat.completions.create(
    model="qwen3-vl:30b",
    messages=[{"role": "user", "content": "Hello"}],
)
```

`AI().openai_base_url()` returns that URL derived from whatever host is configured, so the two styles stay in sync.

Choose whichever fits: the OpenAI SDK if you may swap in a hosted provider later or need ecosystem compatibility; `ai.py` for the lighter dependency, the automatic tool schemas, and the image handling.

Raw HTTP

Nothing is hidden — these are the endpoints underneath:

Endpoint	Purpose
POST /api/chat	Chat, including images and tools
POST /api/generate	Single-prompt completion
POST /api/embed	Embeddings
GET /api/tags	List downloaded models
GET /api/ps	List models in GPU memory
POST /api/pull	Download a model
GET /api/version	Server version / health

```
curl http://gx10-3d18:11434/api/chat -d '{
  "model": "qwen3-vl:30b",
  "stream": false,
  "messages": [{"role": "user", "content": "hello"}]
}'
```

7. Troubleshooting

Symptom	Cause and fix
cannot reach the GX10	The GX10 is asleep or off the tailnet. Check with “tailscale status”; the row for gx10-3d18 should not say offline.
Name does not resolve	MagicDNS is off on this machine. Use the address directly: ROTE_AI_HOST=http://100.88.242.48:11434
First call takes ~30s	Normal: the model is loading into GPU memory. Use ai.warm() at startup or a longer keep_alive.
A 5xx on the first call	The request arrived mid-load. ai.py retries automatically; warming avoids it.
model ... not found	Not downloaded yet. Run: python ai.py --pull
Timeouts on long generations	Raise ROTE_AI_TIMEOUT (seconds), or stream so output arrives incrementally.
Tools are ignored	The model must advertise the tools capability — check python ai.py --check. Also make the docstring describe clearly when to call it.
Answers are inconsistent	Set temperature=0, and seed for repeatability.

Security. Ollama on the GX10 has no authentication, so anything that can reach port 11434 can use the GPU and read any model on it. That is fine while the tailnet is the only route to it. If it is ever bound to a wider interface or forwarded publicly, put an authenticating proxy in front first.